



Riskly(Vue 2 + Nuxt)

1. 프로젝트 개요

- **프로젝트명:** Riskly (암호화폐 온체인 리스크 평가 플랫폼)
- **개발 기간:** 2023.06 ~ 2023.08
- **수행 방식:** 단독 개발
- **담당 범위:** 전체 프론트엔드 개발
- **배포 현황:** 실제 서비스용으로 개발되었으며, 현재는 페이지가 닫힌 상태
- **기술 스택:** Nuxt.js, Vue 2, Vuex, Sass, axios, highcharts-vue, html2canvas, nuxt-i18n, vue-sweetalert2, vue-toastification, swiper

2. 클라이언트 요구사항 및 나의 역할

클라이언트 개요

암호화폐 자산의 온체인(On-Chain) 유동성 리스크를 실시간으로 평가하는 데이터 기반 플랫폼. 다양한 리스크 지표를 시각화된 대시보드와 리서치 아티클로 제공하여 사용자들이 암호화폐 투자에 대한 위험을 더 명확하게 이해하도록 돕는 서비스.

요구사항 요약

- 온체인 리스크 데이터를 기반으로 한 실시간 리스크 점수 대시보드 구현.
- `stETH`, `cbETH` 등 다양한 암호화폐 자산의 리스크 등급을 보여주는 리더보드 구현.
- 다양한 암호화폐 데이터를 라인차트, 컬럼차트, 멀티스택차트, 링차트 등 차트와 보고서로 표현
- 대시보드 차트 데이터를 스크린샷 또는 CSV 파일로 내보내기 기능 구현.

담당 역할

- Nuxt.js 기반의 프론트엔드 프로젝트 아키텍처 설계 및 단독 개발

- 리스크 대시보드, 리더보드, 리서치, 문서, 계정 관리 등 주요 페이지 개발.
- **Highcharts** 라이브러리를 활용한 온체인 리스크 데이터의 차트 시각화 및 데이터 내보내기 기능 구현

3. 기술 스택 및 아키텍처

항목	사용 기술 / 라이브러리
프레임워크	Nuxt.js, Vue 2
상태관리	Vuex
네트워킹	axios
차트/시각화	highcharts-vue, Highcharts, apexcharts, Chart.js
유틸리티	html2canvas, vue-sweetalert2, vue-toastification, moment-timezone, swiper, vue-select
협업 도구	Bitbucket

아키텍처 특징

- **SPA 기반의 데이터 대시보드:** Nuxt.js의 SPA(Single Page Application) 방식을 활용, **axios** 를 통한 비동기 데이터 통신으로 방대한 온체인 데이터를 효율적으로 처리하는 아키텍처 설계.
- **동적 시각화 컴포넌트:** **Highcharts** 를 사용하여 다양한 차트(Area, Bar, Donut, Line 등) 컴포넌트를 개발, 각 컴포넌트의 데이터와 옵션을 동적으로 관리하여 유연한 데이터 시각화 구현.

4. 주요 기능

기능	설명
리스크 대시보드	Risk Rating Score를 비롯한 여러 온체인 리스크 지표를 시간대별로(30d, 90d, 180d, All) 차트로 시각화하는 대시보드 제공.
리스크 리더보드	여러 암호화폐 자산(stETH, cbETH 등)의 리스크 등급(A, B, C, D)과 핵심 메트릭스(Staked/Market share, Peg Premium 등)를 비교 분석하는 리더보드 구현.
데이터 내보내기	대시보드 차트 데이터를 Screenshot 이미지나 Download CSV 파일로 내보내는 기능 구현.

계정 관리	이메일과 비밀번호를 이용한 회원가입, 로그인, 비밀번호 재설정, 프로필 수정 등 사용자 계정 관리 기능 구현.
-------	---

5. 문제 해결 및 기술적 도전

✓ 문제 1: 차트 라이브러리 선정 및 커스텀 기능 구현

다양한 데이터를 시각화하기 위해 여러 차트 라이브러리를 검토하고, 프로젝트 요구사항에 가장 적합한 라이브러리 채택.

→ `vue-chartjs`, `apexcharts` 등 여러 라이브러리를 검토한 뒤, 프로젝트 요구사항에 가장 적합한 `highcharts` 를 채택. `highcharts` 를 사용하여 바 차트, 라인 차트, 스택바 차트 등 다양한 차트 유형을 구현.

✓ 문제 2: 복잡한 데이터 시각화 및 부가 기능 개발

방대한 온체인 데이터를 실시간으로 받아와 다양한 차트로 시각화하고, 이를 이미지나 CSV 파일로 내보내는 기능 구현 도전.

→ **차트 커스텀:** y축 레이블을 사용자 정의 포맷으로 변경하고, `formatter` 함수 내 조건부 처리를 통해 특정 값만 표시하는 커스텀 로직 구현. 막대 차트의 코너 라운딩 구현을 위해 `Highcharts` 11버전의 `plotOptions` 속성 활용.

→ **부가 기능 및 공용 컴포넌트:** 기간 선택, 링크 복사, 스크린샷, CSV 다운로드 등 차트의 부가 기능들을 공용 컴포넌트와 믹스인(`mixin`)으로 분리하여 개발 효율성 및 재사용성 확보. `html2canvas` 라이브러리의 `ignoreElements` 옵션으로 스크린샷 시 불필요한 UI가 포함되는 문제 해결. 수치에 따라 단위를 동적으로 조정하는 공용 함수를 구현, 데이터 표기 가독성 향상.

✓ 문제 3: 동적 UI 구현 및 반응형 레이아웃 처리

스크롤에 따라 사이드바를 고정하고, 데이터 종류가 많은 테이블의 가독성을 유지하며 반응형 레이아웃을 처리하는 도전.

→ **스크롤 반응형 UI:** `position: fixed` 속성과 `window.addEventListener('resize')` 를 활용하여 뷰포트 크기에 따라 사이드바의 `left` 값을 동적으로 계산, 헤더와 독립적으로 스크롤을 따라다니는 고정형 사이드바 구현.

→ **반응형 테이블/차트 디자인:** 테이블 데이터의 가독성 저하 문제를 해결하기 위해, `white-space: pre-line` 속성으로 특정 열의 텍스트를 줄 바꿈하여 모바일 및 태블릿 환경에 최적화된 UI/UX 구현. 모바일 환경에서 차트 툴팁이 잘리는 문제를 해결하기 위해 툴팁 내 값들을 줄 바꿈하여 전체 내용이 표시되도록 처리.

6. 협업 및 커뮤니케이션

- **시각적 데이터 기반의 협업:** 차트 등 시각적인 데이터를 다루는 특성상, 기획자 및 디자이너와 긴밀히 소통하며 데이터의 표현 방식과 UI/UX를 협의.
 - **QA를 통한 서비스 품질 관리:** QA를 통해 발견된 버그를 신속하게 수정하고, 기능 개선 사항을 논의하며 서비스 품질 향상에 기여.
-

7. 개발 결과 및 회고



결과

- **복합적인 데이터 기반 서비스 구축:** 리스크 대시보드, 리더보드, 리서치 등 복합적인 기능을 성공적으로 구현하여 서비스의 핵심 가치를 전달
- **효율적인 데이터 시각화 구현:** `Highcharts` 라이브러리를 활용해 방대한 데이터를 가독성 높은 차트로 시각화하고, 데이터 내보내기 기능으로 사용자 편의성 증대.
- **각종 암호화페 데이터의 효과적인 시각화:** 각종 암호화페 데이터를 차트, 테이블 등으로 반응형과 UI를 효과적으로 구현.

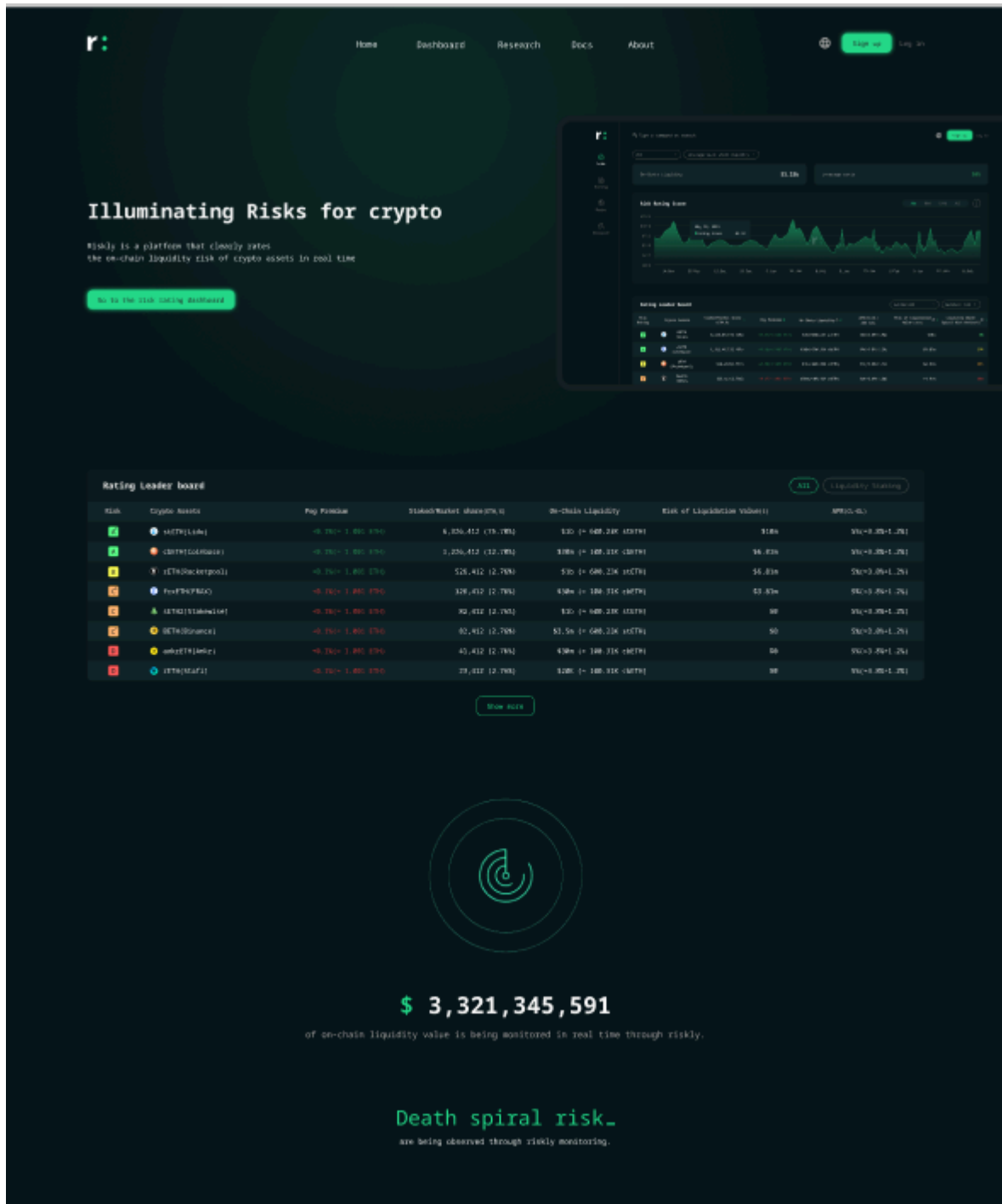


회고

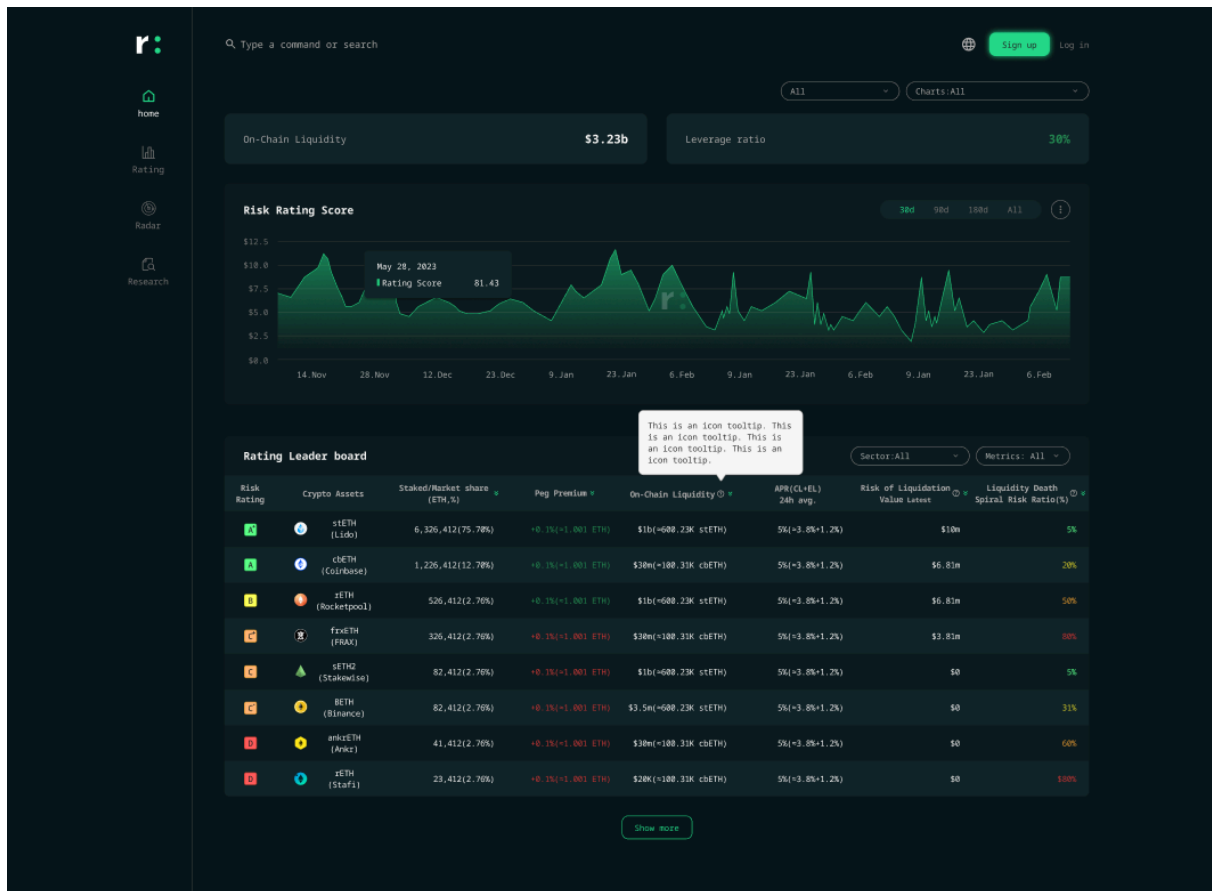
- **데이터 기반 UI/UX 개발 경험:** `Highcharts` 를 사용해 복잡한 데이터를 시각화하고, 차트 데이터를 내보내는 기능 구현을 통해 데이터 기반 서비스 개발 역량 강화.
 - **유기적인 소통의 중요성:** 기획이 미완성된 프로젝트를 기획/디자인 담당자와 지속적으로 소통하며 완성해 나가는 과정을 통해 문제 해결 능력 및 커뮤니케이션 능력 증진.
-

8. 스크린샷 및 데모 영상

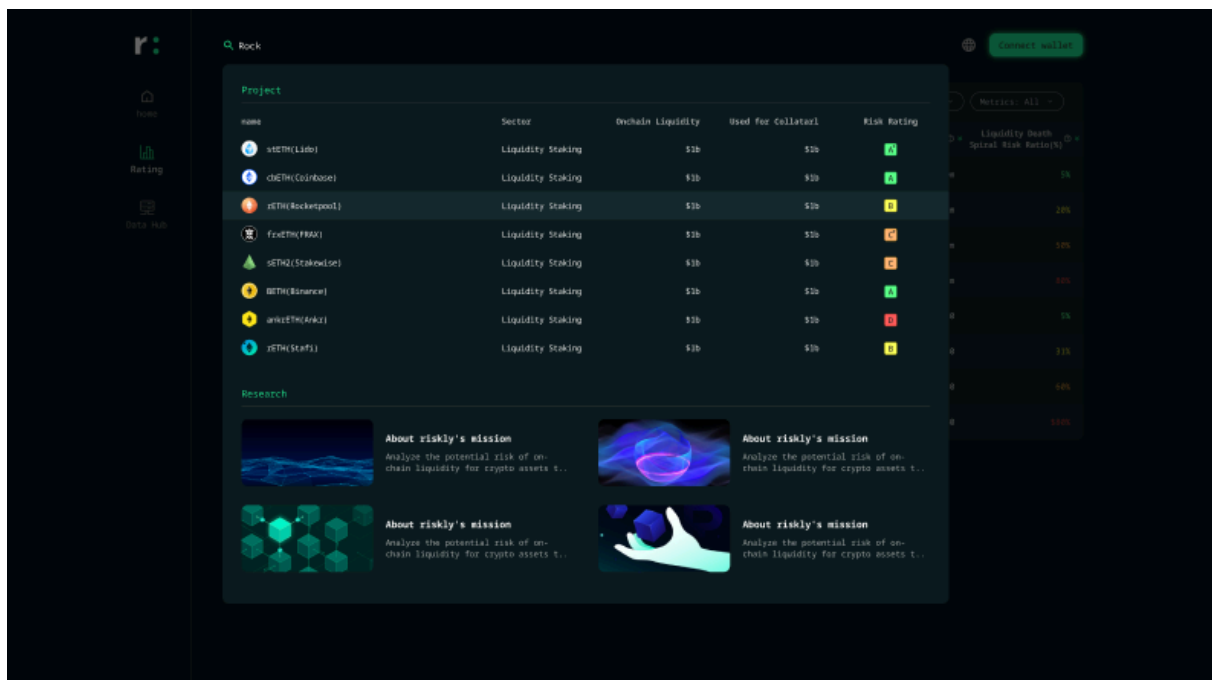
- 앱 주요 화면별 캡처 이미지 (홈 / 기능 A / 기능 B 등)



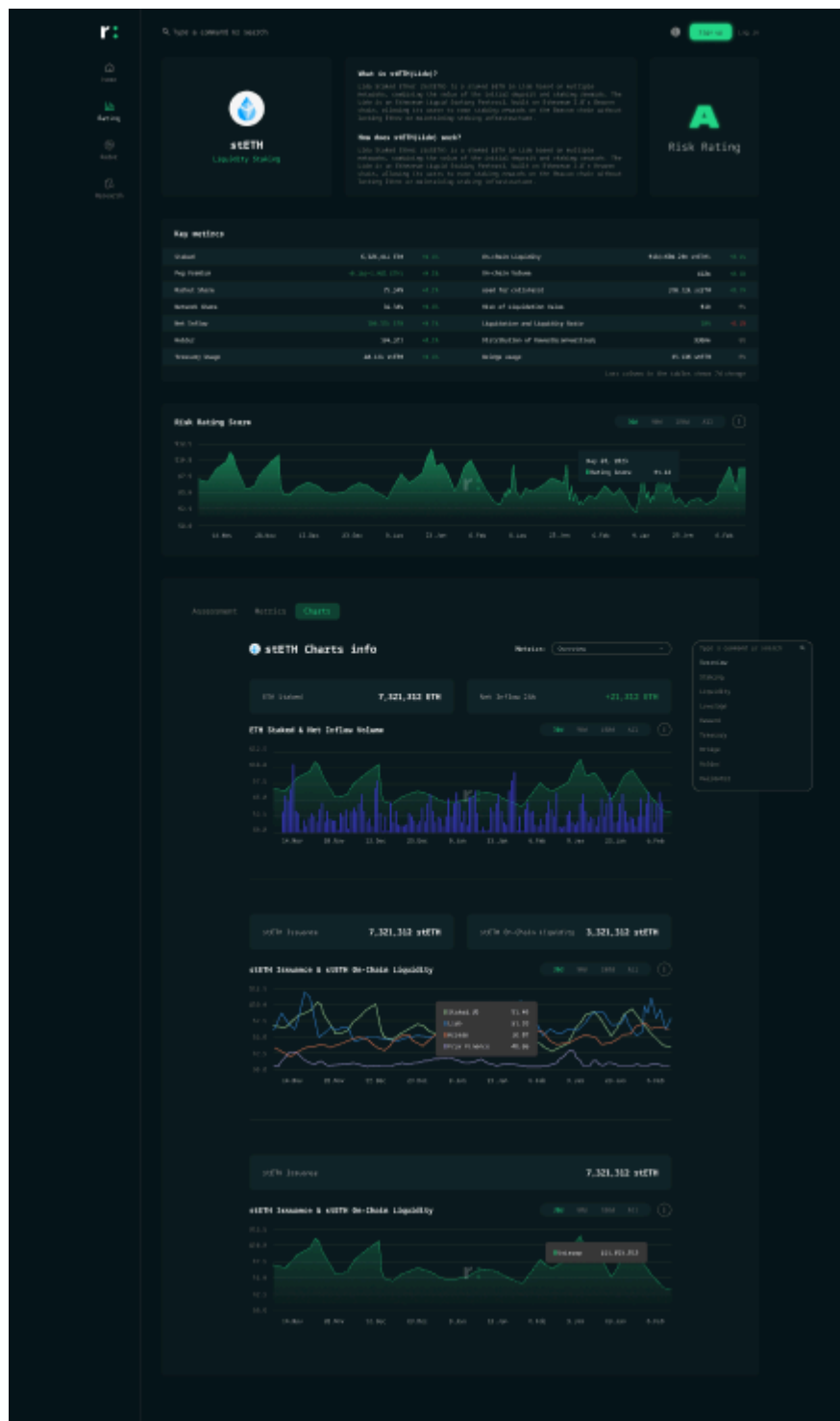
이미지 1. 메인 페이지



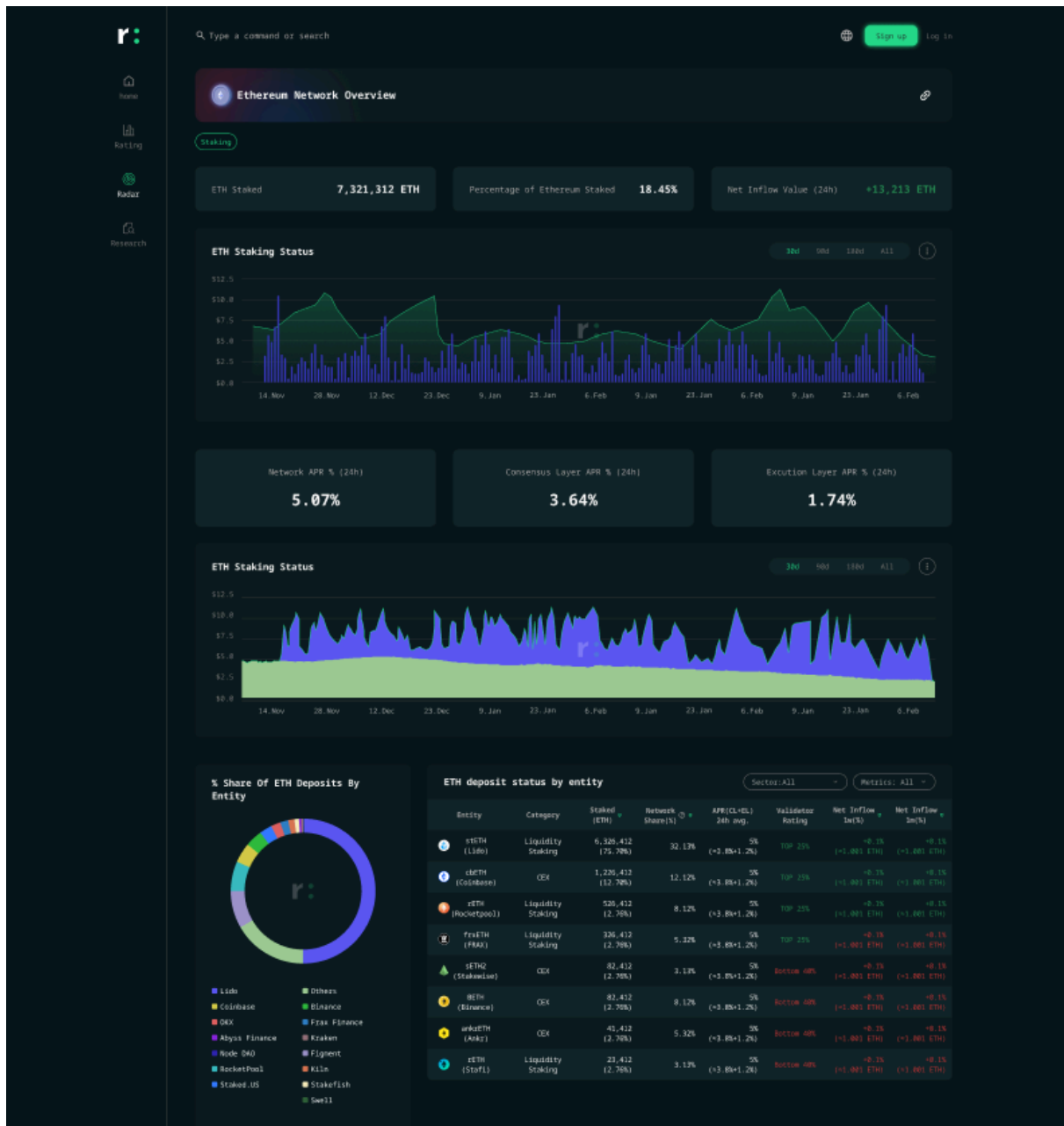
이미지 2. 대시보드 페이지



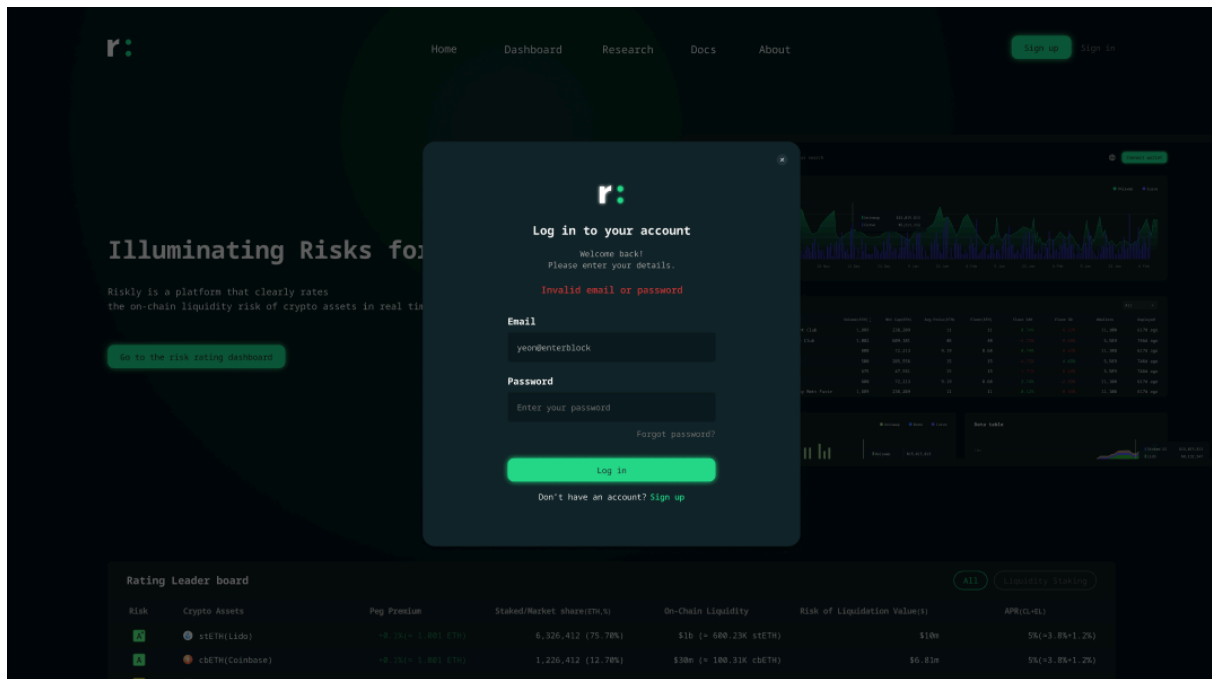
이미지 3. 리서치 페이지(검색화면)



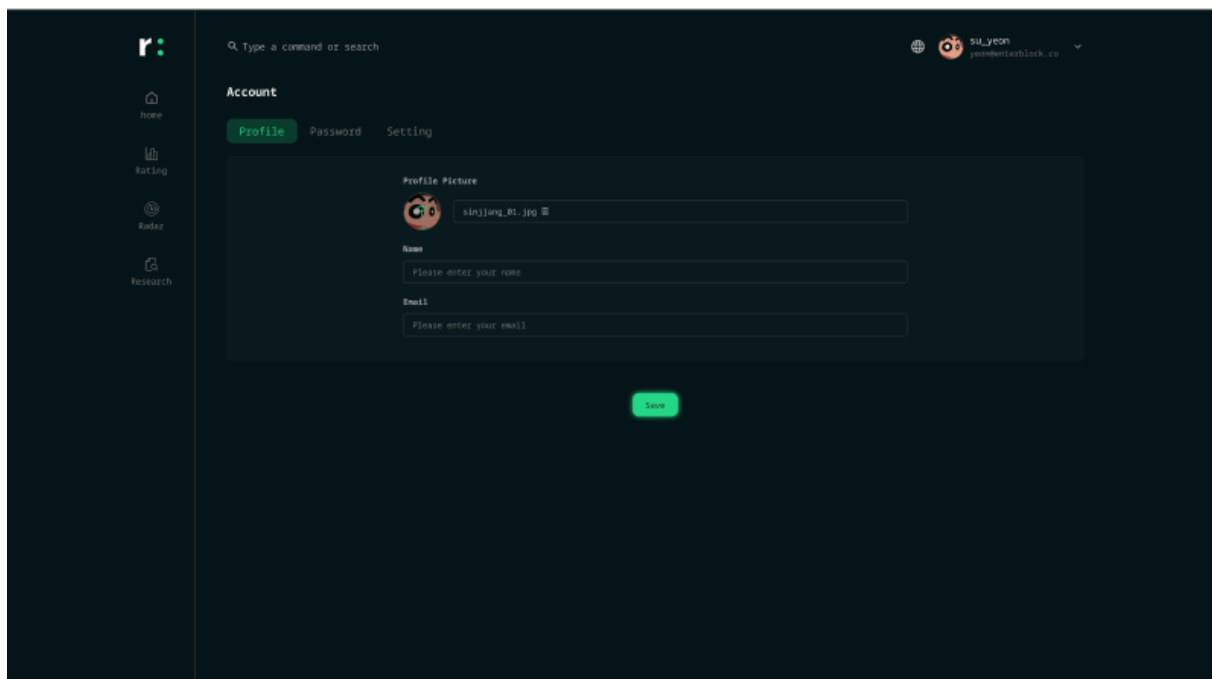
이미지 4. 리서치 상세 페이지



이미지 5. 레이더 상세 페이지



이미지 6. 로그인 페이지



이미지 7. 어카운트 페이지

9. 표지 이미지

